

# HARDWARE

V předchozí kapitole o kompilátorech jsme sledovali, jak se zdrojový program postupně převádí do podoby, kterou lze vykonat. Tím však příběh programu nekončí. Výsledné strojové instrukce musí načíst procesor, určit jejich význam, získat potřebná data a provést požadované operace. K tomu procesor potřebuje registry, různé druhy paměti a cesty, po nichž se mezi jednotlivými částmi počítače přenášejí data, adresy a řídicí signály.

V této kapitole se proto podíváme na základní části počítače z pohledu vykonávání programu. Nejde nám o podrobný elektrotechnický popis jednotlivých součástek. Chceme především pochopit, proč nestačí říci, že program „běží v procesoru“, proč počítač používá několik různých druhů paměti a jak konstrukce paměťového subsystému ovlivňuje rychlost programu. Začneme obecným schématem počítače, poté rozebereme procesor a nakonec se budeme věnovat pamětem od registrů až po dlouhodobá úložiště.

## 1 Základní části počítače

Slovem *hardware* označujeme fyzické součásti počítače: procesor, paměťové čipy, základní desku, úložiště, klávesnici, displej a další zařízení. Samotný hardware však neurčuje, jakou konkrétní úlohu má počítač řešit. Potřebný postup popisuje *software*, tedy programy a data, s nimiž tyto programy pracují. Stejný počítač proto může chvíli zobrazovat webovou stránku, poté překládat program a nakonec přehrávat video. Hardware zůstává stejný, mění se pouze instrukce, které právě vykonává.

Při velmi hrubém pohledu můžeme činnost počítače rozdělit mezi následující části:

- **datová část** provádí výpočty a zpracovává data;
- **řídicí část** určuje, co a v jakém pořadí se má stát;
- **paměť** uchovává programy, jejich instrukce, vstupní data a mezivýsledky;
- **vstupní zařízení** předávají počítači data z okolí;
- **výstupní zařízení** předávají výsledky uživateli nebo jinému zařízení.

Datová a řídicí část jsou v běžném počítači součástí *procesoru*, často označovaného zkratkou CPU z anglického *Central Processing Unit*. Procesor je sice místem, kde se instrukce skutečně provádějí, bez paměti a vstupně/výstupních zařízení by však neměl odkud instrukce ani data získat a kam uložit výsledky. Jednotlivé části proto nelze chápat jako izolované krabičky; při vykonávání programu spolu neustále komunikují.

Na obrázku 1 je zachycen typický tok informací. Vstupní zařízení dodá data, procesor je podle instrukcí programu zpracuje a výsledek může předat výstupnímu zařízení. Během tohoto procesu se instrukce i data přesouvají mezi procesorem a pamětí. Rozdělení na vstup a výstup přitom není vždy absolutní. Například síťová karta data přijímá i odesílá a dotyková obrazovka současně zobrazuje výstup a zaznamenává vstup uživatele.

V této kapitole se zaměříme hlavně na procesor a paměťový subsystém, tedy na části, které přímo souvisejí s vykonáváním programu. Vstupně/výstupní zařízení budeme uvažovat pouze na úrovni potřebné k pochopení celkového schématu.

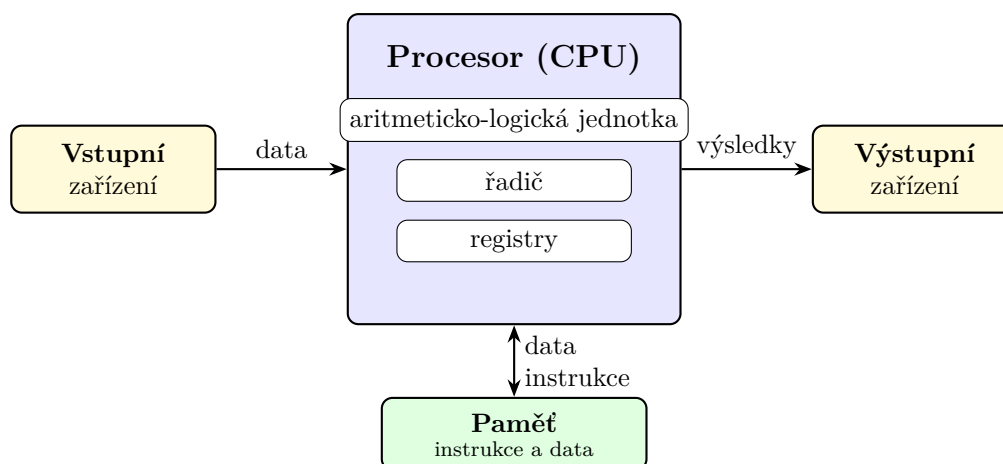


Figure 1: Zjednodušené schéma základních částí počítače.

## 1.1 Data, instrukce a jejich význam

Uvnitř počítače jsou informace reprezentovány pomocí bitů. Samotná posloupnost bitů však ještě neříká, co představuje. Stejný vzor může být podle okolností interpretován jako celé číslo, část obrázku, znak textu, adresa v paměti nebo strojová instrukce. Význam tedy nevychází pouze z uložených bitů, ale také z toho, jak s nimi právě pracuje procesor a jaký formát programu očekává.

Tento princip je důležitý i pro programování. Proměnná určitého datového typu určuje, jak má program uloženou hodnotu chápat, zatímco instrukce určuje, jaká operace se má s hodnotou provést. Na úrovni hardware jsou však obě informace stále reprezentovány bity a pro jejich přenos se používají stejné základní fyzikální prostředky.

Při hledání chyby v programu je užitečné rozlišovat mezi uloženou hodnotou a její interpretací. Bity mohou být přeneseny nebo uloženy správně, ale program je může vyložit v nesprávném formátu.

## 2 Uložení programu a dat

### 2.1 Von Neumannova architektura

Základní představu o uspořádání počítače poskytuje *von Neumannova architektura*. Jejím podstatným rysem je, že program i data jsou uloženy ve společné paměti. Procesor z ní načítá strojové instrukce a stejným způsobem přistupuje také k hodnotám, které mají být zpracovány. Instrukce určují, jaká operace se má provést, odkud se mají získat operandy a kam se má uložit výsledek.

Tato myšlenka se někdy označuje jako *princip uloženého programu*. Program není pevně zabudován do zapojení počítače, ale je uložen v paměti podobně jako ostatní data. Změnou obsahu paměti tak můžeme změnit činnost počítače, aniž bychom museli přestavět jeho hardware. Právě tato univerzálnost je jedním z důvodů, proč se daný princip stal základem obecných počítačů.

Paměť v tomto modelu sama nerozlišuje, zda uložené bity představují data, nebo instrukce. Rozhoduje až způsob, jakým je procesor použije. Procesor opakovaně načte instrukci z paměti, zjistí její význam

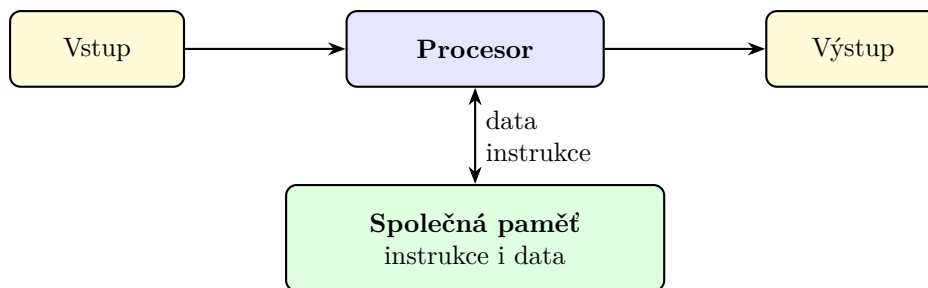


Figure 2: Základní uspořádání von Neumannovy architektury.

a provede ji. Pokud instrukce pracuje s daty uloženými v paměti, musí procesor obvykle provést další paměťové přenosy. Zjednodušeně tak dostáváme stále se opakující cyklus

načtení instrukce  $\rightarrow$  dekodování  $\rightarrow$  provedení.

Jeho podrobnější podobu rozebereme v podsekcí 3.2.

Společná cesta pro instrukce i data přináší jednoduché a univerzální uspořádání, ale současně může omezovat rychlost. Procesor musí s pamětí komunikovat při načítání instrukcí i při čtení a zápisu dat. Pokud je procesor schopen pracovat rychleji, než mu paměť dodává potřebné informace, část času pouze čeká. Toto omezení se označuje jako *von Neumannovo úzké hrdlo*.

Rychlost počítače neurčuje pouze počet operací, které dokáže procesor provést za sekundu. Stejně důležité je, zda procesor dostává instrukce a data dostatečně rychle. Právě proto bude později tak významná cache a celá paměťová hierarchie.

## 2.2 Harvardská architektura

*Harvardská architektura* používá oddělenou paměť pro instrukce a oddělenou paměť pro data. Obě paměti mohou mít vlastní adresový prostor i vlastní přenosové cesty. Procesor pak může současně načítat další instrukci a přenášet data potřebná pro právě prováděný výpočet. Paměť instrukcí navíc může mít jiné vlastnosti než paměť dat, protože se od ní může očekávat jiný způsob používání.

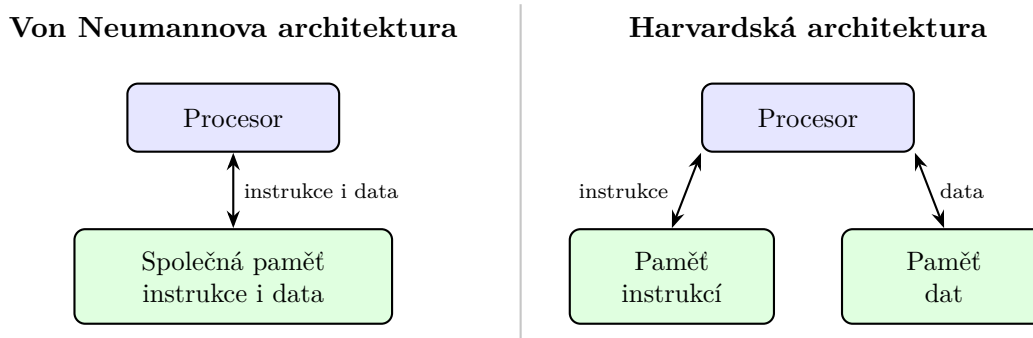


Figure 3: Porovnání von Neumannovy a harvardské architektury.

Rozdíl obou přístupů zachycuje obrázek 3. U von Neumannovy architektury vede mezi procesorem a společnou pamětí jedna logická cesta pro instrukce i data. V harvardském uspořádání má procesor k dispozici dvě oddělené cesty. To však neznamena, že každý reálný počítač musí přesně odpovídat jednomu z těchto dvou schémat.

V praxi se často používá *modifikovaná harvardská architektura*. Z pohledu programu může existovat jeden společný adresový prostor, zatímco uvnitř procesoru jsou oddělené například instrukční a datová cache. Počítač se tedy navenek chová podobně jako von Neumannův, ale v částech, kde je výhodné souběžné načítání, využívá prvky harvardského uspořádání.

Von Neumannova a harvardská architektura jsou především užitečné modely. Skutečné procesory obsahují mnoho dalších mechanismů a jejich vnitřní uspořádání bývá kombinací více myšlenek.

### 3 Procesor

Procesor vykonává strojové instrukce programu. Nejde však o jedinou součástku provádějící všechny činnosti stejným způsobem. Uvnitř procesoru spolupracuje několik částí, z nichž každá má jinou úlohu. Pro náš zjednodušený model budou nejdůležitější aritmeticko-logická jednotka, řadič a registry.

- **Aritmeticko-logická jednotka (ALU)** provádí aritmetické a logické operace. Může například sčítat čísla, porovnávat hodnoty nebo provádět bitové operace.
- **Řadič** organizuje činnost procesoru. Na základě právě načtené instrukce vytváří řídicí signály, kterými určuje, odkud se mají získat operandy, jakou operaci má provést ALU a kam se má uložit výsledek.
- **Registry** uchovávají malé množství právě používaných dat přímo uvnitř procesoru. Přístup k nim je výrazně rychlejší než přístup do hlavní paměti.

Skutečný procesor obsahuje ještě řadu dalších částí, například jednotky pro práci s desetinnými čísly, mechanismy pro předvídání větvení, několik úrovní cache nebo jednotky schopné provádět jednu operaci nad více hodnotami současně. Základní vztah mezi řadičem, výpočetními jednotkami, registry a pamětí však zůstává užitečný i při popisu podstatně složitějších procesorů.

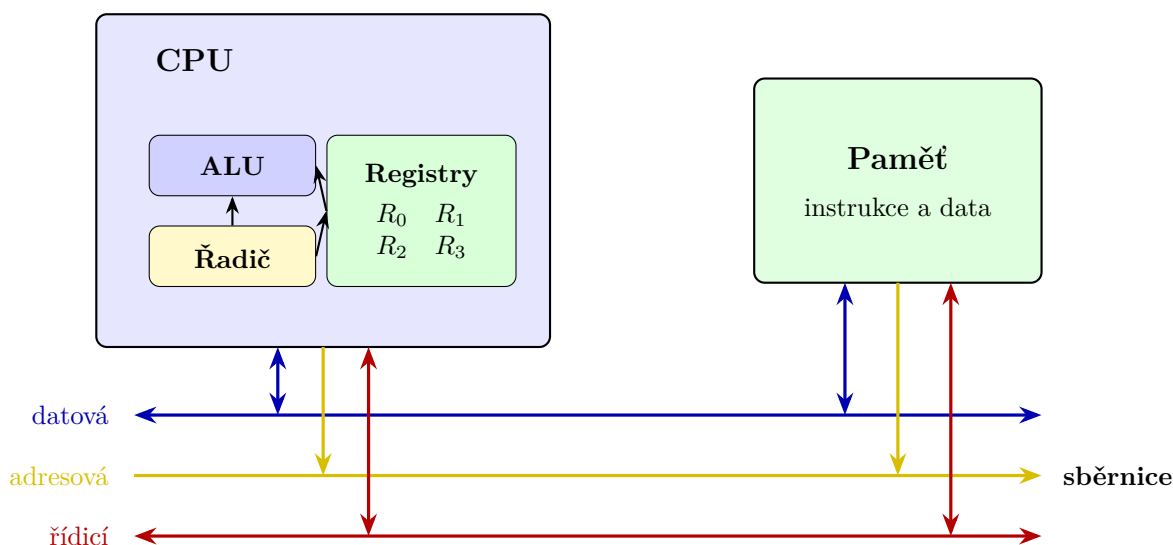


Figure 4: Zjednodušená vnitřní struktura procesoru a jeho spojení s pamětí.

### 3.1 Sběrnice

Procesor, paměť a další zařízení potřebují přenášet různé druhy informací. Soubor vodičů a pravidel, podle nichž se po nich informace přenášejí, nazýváme *sběrnice*. Na obrázku 4 rozlišujeme tři logické skupiny signálů:

- **datová sběrnice** přenáší samotná data a instrukce;
- **adresová sběrnice** určuje paměťové místo nebo zařízení, s nímž chce procesor komunikovat;
- **řídící sběrnice** přenáší signály určující například směr přenosu, požadavek na čtení či zápis nebo informaci o dokončení operace.

Uvažujme například čtení hodnoty z paměti. Procesor nejprve předá na adresovou sběrnici adresu požadovaného místa a řídícím signálem oznámí, že chce číst. Paměť vyhledá odpovídající obsah a předá jej po datové sběrnici zpět procesoru. Při zápisu je postup podobný, pouze procesor zároveň odešle hodnotu a řídícím signálem určí, že se má obsah paměti změnit.

Rozdělení na datovou, adresovou a řídící sběrnici popisuje význam přenášených signálů. Ve skutečném zařízení nemusí každé skupině odpovídat samostatná sada vodičů; některé fyzické spoje mohou v různých okamžicích přenášet různé druhy informací.

### 3.2 Instrukční cyklus

Program je pro procesor posloupností strojových instrukcí uložených v paměti. Každá instrukce musí být načtena, rozpoznána a provedena. Jednotlivé procesory se v podrobnostech liší, ale základní průběh lze popsat následujícími kroky:

1. **Čtení instrukce (*fetch*)**. Procesor získá z paměti instrukci, na kterou ukazuje čítač instrukcí.
2. **Dekódování instrukce (*decode*)**. Řadič určí, jakou operaci instrukce požaduje a kde se nacházejí její operandy.
3. **Čtení operandů (*operand fetch*)**. Potřebné hodnoty se získají z registrů, případně z paměti.
4. **Provedení instrukce (*execute*)**. Příslušná výpočetní jednotka provede požadovanou operaci.
5. **Uložení výsledku (*write back*)**. Výsledek se zapíše do registru nebo do paměti.

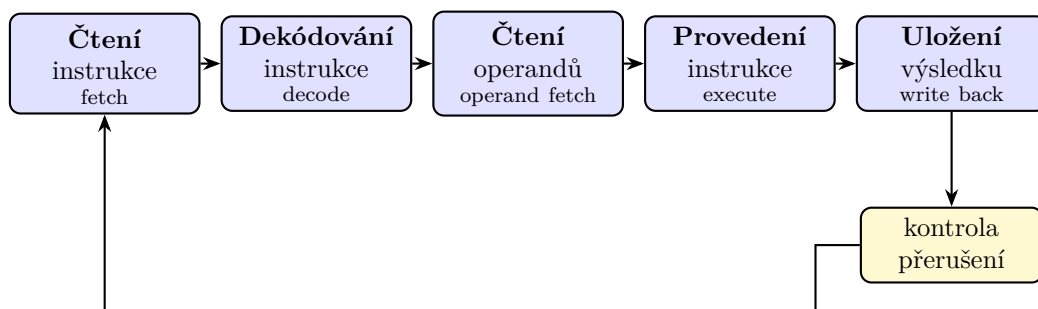


Figure 5: Zjednodušený instrukční cyklus procesoru.

Představme si například instrukci, která má sečíst hodnoty registrů  $R_0$  a  $R_1$  a výsledek uložit do registru  $R_2$ . Procesor instrukci nejprve načte a dekoduje. Poté přečte hodnoty obou zdrojových registrů, předá je ALU, nechá provést součet a výsledek zapíše do cílového registru. Následně může pokračovat další instrukcí.

Obrázek 5 vytváří dojem, že procesor dokončí všechny kroky jedné instrukce a teprve potom začne načítat další. Takový postup je možný, moderní procesory však jednotlivé fáze často překrývají. Zatímco jedna instrukce se provádí, další se může dekódovat a ještě další načítat. Tento způsob zpracování se nazývá *zřetězení* neboli *pipeline*. Překrývání fází zvyšuje počet instrukcí, které lze zpracovat za určitou dobu, přestože doba průchodu jedné instrukce všemi fázemi se nemusí zkrátit.

Činnost procesoru je řízena hodinovým signálem. Jeho frekvence udává počet taktů za sekundu, nikoli přímo počet dokončených instrukcí. Některá instrukce může vyžadovat více taktů, více instrukcí se může zpracovávat souběžně a procesor může čekat na data z paměti. Samotná hodnota frekvence proto nestačí k porovnání výkonu různých procesorů.

### 3.3 Přerušení

Běžící program není jediným zdrojem událostí, na které musí procesor reagovat. Klávesnice může oznámit stisk klávesy, úložiště dokončení přenosu nebo časovač uplynutí stanoveného intervalu. K takovým situacím slouží *přerušení*. Procesor po dokončení vhodného kroku uloží informace potřebné pro pozdější pokračování programu a přejde k obsluze vzniklé události. Po jejím vyřízení se může k původnímu programu vrátit.

Přerušení umožňuje, aby procesor nemusel každé zařízení neustále kontrolovat. Zařízení jej samo upozorní ve chvíli, kdy potřebuje obsluhu. Operační systém pak rozhodne, jaká část programu má událost zpracovat a kdy může pokračovat přerušovaný výpočet.

### 3.4 Registry

Registry jsou velmi malé a rychlé paměti přímo uvnitř procesoru. Uchovávají operandy právě prováděných instrukcí, adresy, mezivýsledky i informace o stavu výpočtu. Instrukce obvykle pracují s registry přímo, zatímco hodnotu z hlavní paměti je nejprve potřeba do registru načíst a výsledek případně později zapsat zpět.

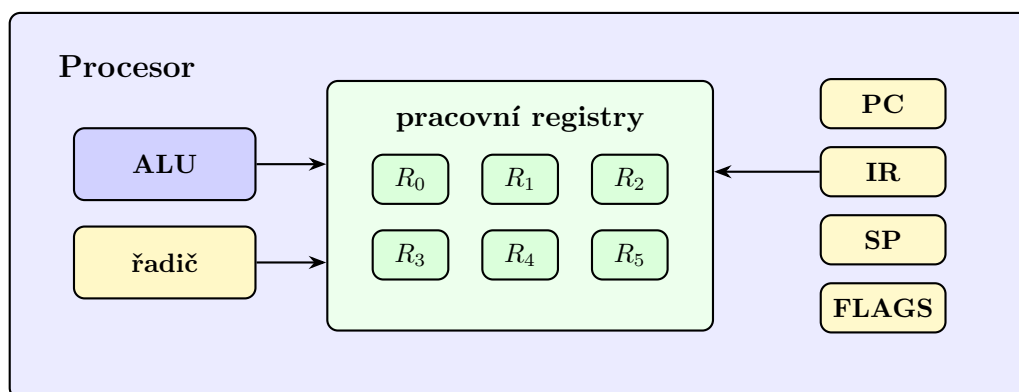


Figure 6: Pracovní a vybrané speciální registry v procesoru.

Vedle pracovních registrů, jejichž význam určuje právě prováděný program, se setkáme také s registry se zvláštní úlohou:

- **PC** (*program counter*) obsahuje adresu následující instrukce, která se má načíst. Po běžné instrukci se posune vpřed, zatímco instrukce skoku do něj může uložit jinou adresu.
- **IR** (*instruction register*) uchovává právě načtenou nebo prováděnou instrukci.

- **SP** (*stack pointer*) ukazuje na vrchol zásobníku v paměti. Zásobník se využívá například při volání funkcí.
- **FLAGS** uchovává příznaky výsledku poslední operace, například zda byl výsledek nulový, záporný nebo zda došlo k přetečení.

Konkrétní názvy, počty a přesný význam registrů závisí na instrukční sadě procesoru. Obrázek 6 proto nepředstavuje schéma jednoho konkrétního modelu, ale obecnou ilustraci rolí, které registry mohou zastávat.

Registry nejsou určeny k dlouhodobému uložení velkého množství dat. Jejich hlavní výhodou je rychlost a přímá dostupnost pro instrukce procesoru; za tuto výhodu platíme velmi malou kapacitou.

## 4 Vnitřní paměti

Procesor dokáže provádět operace velmi rychle, ale pro většinu instrukcí potřebuje data. Kdyby musel při každé operaci čekat na hlavní paměť, zůstala by velká část jeho výkonu nevyužita. Počítač proto nepoužívá jediný druh paměti. Mezi procesorem a hlavní pamětí se nachází cache a přímo uvnitř procesoru registry, které jsme již popsali v podsekcí 3.4.

### 4.1 Cache

Cache je malá a velmi rychlá paměť, která uchovává kopie vybraných dat a instrukcí z pomalejší paměti. Když procesor požaduje určitou hodnotu, nejprve se ověří, zda její kopie není v cache. Pokud se hledaná informace v cache nachází, nastane *cache hit* a procesor ji získá rychle. Pokud se v cache nenachází, nastane *cache miss*; data je potřeba načíst z nižší, pomalejší úrovně paměti a současně se obvykle uloží do cache pro případ dalšího použití.

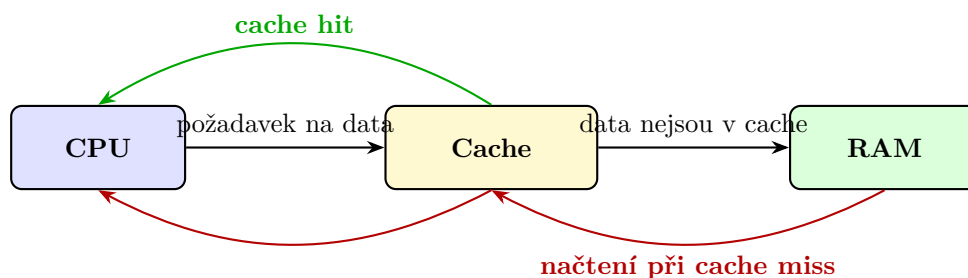


Figure 7: Obsluha požadavku procesoru pomocí cache.

Cache není pouhým seznamem jednotlivých naposledy použitých hodnot. Data se mezi paměťovými úrovněmi přenášejí po blocích, kterým se říká *cache line*. Jestliže procesor požádá o jeden bajt nebo jedno číslo, do cache se spolu s ním načte také okolní část paměti. Tento postup je výhodný díky dvěma často pozorovaným vlastnostem programů:

- **časová lokalita** znamená, že nedávno použitá data nebo instrukce budou pravděpodobně brzy použity znovu;
- **prostorová lokalita** znamená, že po přístupu k určitému místu budou pravděpodobně brzy potřeba také okolní místa.

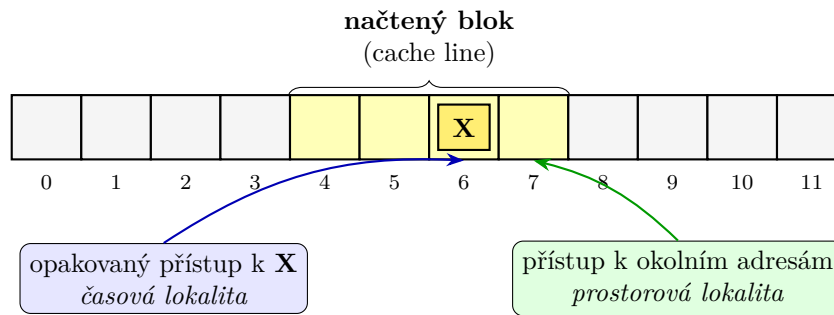


Figure 8: Příklad prostorové a časové lokality při práci s pamětí.

Sekvenční průchod polem obvykle vykazuje dobrou prostorovou lokalitu, protože program postupně čte sousední paměťová místa. Opakované používání stejné proměnné uvnitř cyklu zase vykazuje časovou lokalitu. Naopak přístupy k velkému množství vzdálených adres v nepravidelném pořadí mohou vést k častým výpadkům cache a výpočet se zpomalí, i když počet provedených aritmetických operací zůstane stejný.

Procesory obvykle používají několik úrovní cache. Cache L1 je nejmenší a nejbližší výpočetním jednotkám, další úrovně L2 a L3 bývají postupně větší, ale také pomalejší. Některé úrovně mohou být rozděleny na instrukční a datovou cache, jiné mohou být sdíleny několika jádry procesoru. Smyslem všech úrovní je zachytit co největší část požadavků dříve, než bude nutné přistoupit do výrazně pomalejší hlavní paměti.

Dva programy se stejnou časovou složitostí mohou na skutečném počítači běžet různě rychle. Jedním z důvodů je odlišný způsob přístupu k paměti: algoritmus s lepší lokalitou dokáže využít cache účinněji.

## 4.2 Hlavní paměť RAM

RAM z anglického *Random Access Memory* je hlavní pracovní paměť počítače. Ukládají se do ní právě běžící programy, jejich data a mezivýsledky výpočtů. Je rychlejší než dlouhodobá úložiště, ale pomalejší než cache a registry. RAM je současně *volatilní*: bez napájení se její obsah ztratí.

Označení *random access* neznamená, že by paměť vracela náhodná data. Vyjadřuje, že k paměťovým místům lze přistupovat přímo podle jejich adresy a není potřeba postupně procházet všechna předchozí místa. Z pohledu programu se hlavní paměť chová jako dlouhá lineární posloupnost adresovatelných míst. V běžném počítači adresa obvykle označuje jeden bajt.

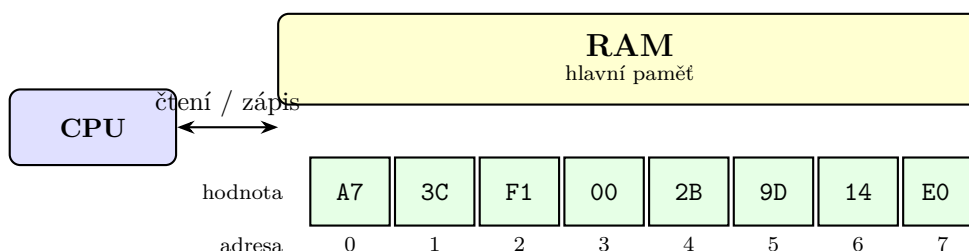


Figure 9: Lineární pohled programu na adresovanou paměť.

Na obrázku 9 jsou na adresách 0 až 7 uloženy osmibitové hodnoty zapsané v šestnáctkové soustavě.



Chce-li procesor přečíst hodnotu z adresy 3, předá tuto adresu paměti a obdrží uloženou hodnotu 00. Větší datový objekt, například 32bitové celé číslo, obvykle zabírá několik sousedních bajtů. Instrukční sada a program pak určují, jak se mají tyto bajty společně interpretovat.

### 4.3 Fyzické uspořádání RAM

Lineární posloupnost adres je rozhraním určeným pro procesor a program. Uvnitř paměťového čipu nejsou buňky nutně uspořádány v jediné řadě, ale tvoří matice řádků a sloupců. Část adresy se použije k výběru řádku a další část k výběru sloupce. Řádkový a sloupcový dekodér tak určí buňku nebo skupinu buněk, s nimiž se má pracovat.

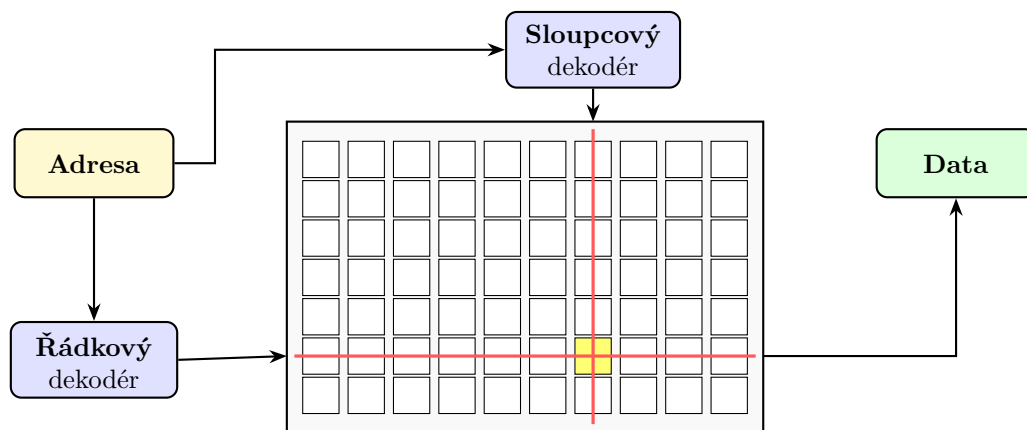


Figure 10: Zjednodušené fyzické uspořádání paměťových buněk do matice.

Hlavní paměť počítače je obvykle tvořena pamětí DRAM. Jedna její buňka uchovává bit pomocí elektrického náboje, který postupně slábne, a proto se musí pravidelně obnovovat. Cache se naproti tomu obvykle vytváří z paměti SRAM. Ta nepotřebuje pravidelné obnovování a je rychlejší, ale jedna buňka zabírá více místa a její výroba je dražší. Proto se SRAM hodí pro malé rychlé cache, zatímco DRAM umožňuje vyrobit hlavní paměť s mnohem větší kapacitou.

| Vlastnost             | SRAM  | DRAM         |
|-----------------------|-------|--------------|
| Typické použití       | cache | hlavní paměť |
| Rychlost              | vyšší | nižší        |
| Hustota buněk         | nižší | vyšší        |
| Potřeba obnovování    | ne    | ano          |
| Uchování bez napájení | ne    | ne           |

Table 1: Základní porovnání pamětí SRAM a DRAM.

SRAM i DRAM jsou volatilní. Rozdíl v potřebě obnovování neznámá, že by SRAM udržela data po vypnutí počítače; pouze za běžného napájení nepotřebuje periodicky obnovovat náboj paměťových buněk.

## 5 Externí paměti a úložiště

Hlavní paměť uchovává data pouze po dobu, kdy je počítač napájen. Programy, dokumenty a další soubory proto potřebujeme ukládat do nevolatilní paměti, jejíž obsah zůstane zachován i po vypnutí.

Mezi nejběžnější dlouhodobá úložiště patří pevné disky HDD a disky SSD. Obě zařízení poskytují operačnímu systému adresovatelný prostor rozdělený do bloků, ale fyzikální princip jejich fungování je odlišný.

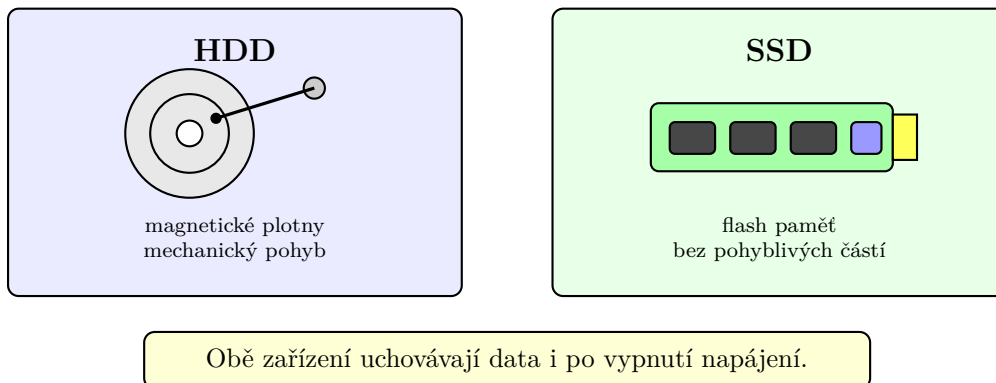


Figure 11: Základní konstrukční rozdíl mezi HDD a SSD.

HDD ukládá data magneticky na rotující plotny a používá pohyblivé čtecí a zapisovací hlavy. SSD ukládá data elektronicky do buněk flash paměti a neobsahuje části, které by se při běžném čtení mechanicky pohybovaly. Díky tomu má SSD zpravidla podstatně rychlejší náhodný přístup, je odolnější vůči otřesům a pracuje tiše. HDD naopak často nabízí velkou kapacitu za nižší cenu.

## 5.1 Pevný disk HDD

Uvnitř HDD se nachází jedna nebo více kruhových ploten upevněných na společném vřetenu. Povrch každé plotny je pokryt magnetickou vrstvou. Plotny se při provozu otáčejí a nad jejich povrchem se pohybují čtecí a zapisovací hlavy umístěné na společném rameni. Hlava se povrchu za běžného provozu nedotýká; změny magnetizace povrchu používá ke čtení a zápisu bitů.

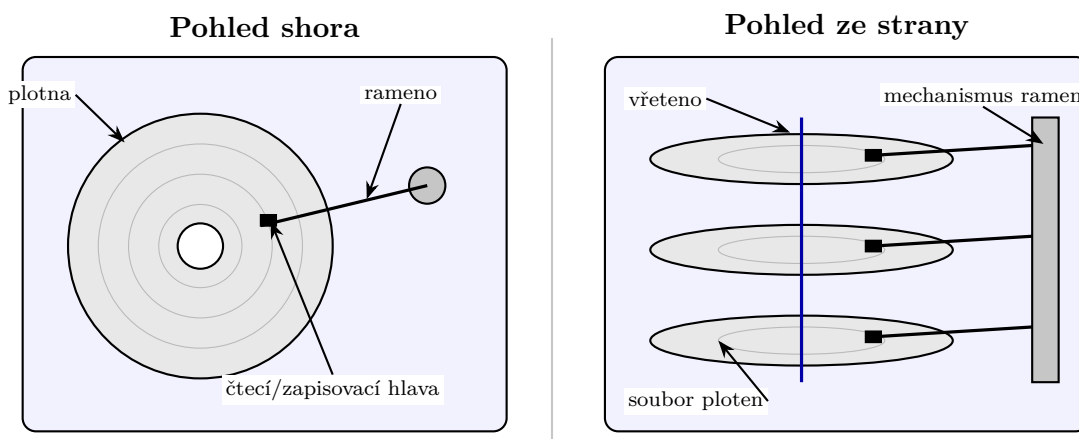
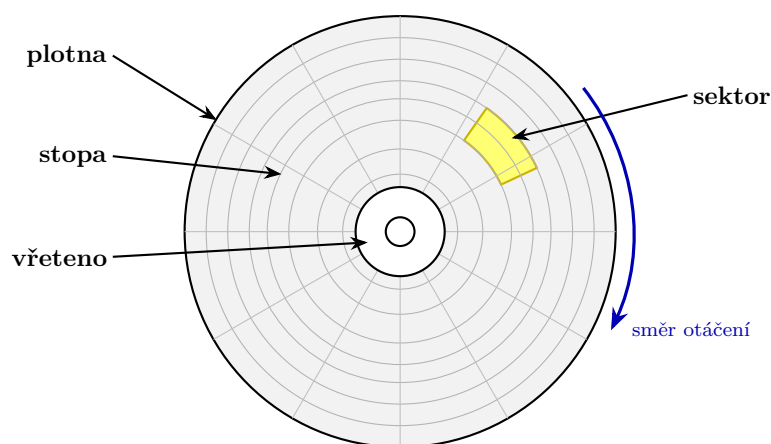


Figure 12: Zjednodušený pohled na vnitřní strukturu HDD shora a ze strany.

Data jsou na povrchu plotny uspořádána do soustředných *stop*. Každá stopa se dále dělí na *sektory*. Stopy ve stejné vzdálenosti od středu na různých površích ploten tvoří *cylindr*. V moderním disku se operační systém nemusí zabývat přesnou fyzickou polohou dat; řadič disku převádí logická čísla bloků na konkrétní místa uvnitř zařízení. Geometrický model je však stále užitečný k pochopení výkonu HDD.

Při čtení bloku musí disk nejprve přesunout rameno nad správnou stopu. Doba tohoto pohybu se označuje jako *doba vystavení* neboli *seek time*. Poté disk čeká, než se požadovaný sektor otočí pod hlavu; vzniká



Povrch plotny je rozdělen na soustředné **stopy**; každá stopa se dále dělí na **sektory**.

Figure 13: Stopy a sektory na jedné plotně HDD.

*rotační zpoždění.* Teprve potom lze data skutečně přenést. Přesun hlavy a čekání na natočení plotny jsou mechanické operace, a proto jsou ve srovnání s elektronickými operacemi pomalé.

Sekvenční čtení velkého souboru je pro HDD výhodné, protože sousední sektory procházejí pod hlavou postupně a rameno se nemusí často přesouvat. Naopak mnoho malých požadavků na vzdálená místa disku vede k opakovanému pohybu hlav a výkon výrazně klesá. Rozdíl mezi sekvenčním a náhodným přístupem je proto u HDD zvláště výrazný.

Při prudkém nárazu může dojít k poškození mechanických částí HDD nebo povrchu plotny. Běžná záloha proto nemá být uložena pouze na druhém oddílu téhož fyzického disku; porucha zařízení by mohla zasáhnout původní data i jejich „zálohu“.

## 5.2 Disk SSD

SSD neukládá data magneticky, ale do buněk nevolatilní flash paměti. Kromě paměťových čipů obsahuje řadič, který přijímá požadavky počítače, vyhledává fyzické umístění dat, opravuje některé chyby a rozděluje zápisy mezi paměťové buňky. Operační systém přitom stejně jako u HDD pracuje s logickými bloky a nemusí znát konkrétní uspořádání buněk uvnitř zařízení.

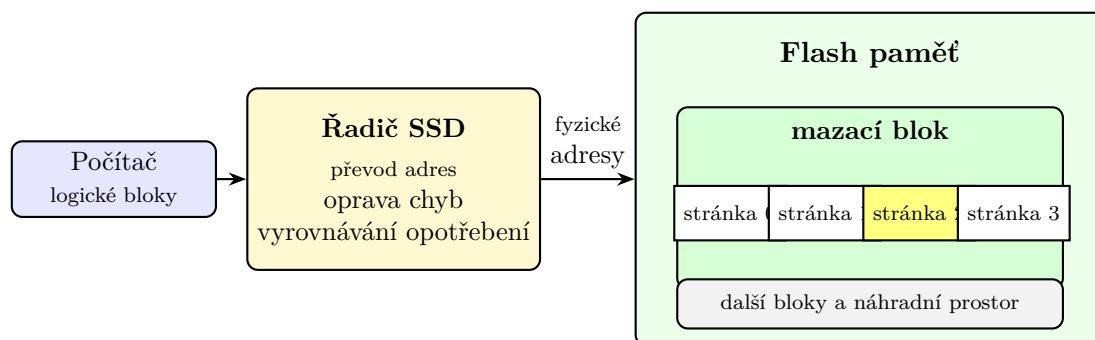


Figure 14: Zjednodušené části SSD a organizace flash paměti.

Flash paměť čte a zapisuje data po *stránkách*, ale mazání probíhá po větších *blocích* obsahujících více stránek. Již zapsanou stránku proto nelze libovolně přepsat stejným způsobem jako bajt v RAM. Řadič obvykle zapíše novou verzi dat na jiné volné místo, změní svou převodní tabulku a později uvolní bloky obsahující již neplatná data.

Každá paměťová buňka snese pouze omezený počet mazacích cyklů. Řadič proto používá *vyrovnávání opotřebení*, kterým se snaží rozložit zápisy mezi různé fyzické bloky. Současně může mít SSD část kapacity vyhrazenou jako náhradní prostor. Tyto mechanismy prodlužují životnost zařízení a umožňují skrýt složitější vlastnosti flash paměti za jednoduché rozhraní logických bloků.

SSD nemá mechanické vyhledávání stopy ani rotační zpoždění, a proto je jeho náhodný přístup podstatně rychlejší než u HDD. Přesto ani SSD není stejně rychlé jako RAM a při zápisu musí jeho řadič řešit omezení flash paměti.

### 5.3 Porovnání HDD a SSD

Volba úložiště závisí na požadované kapacitě, rychlosti, ceně i způsobu použití. Pro systémový disk a často spouštěné aplikace je obvykle výhodné SSD, protože zkracuje čekání při mnoha menších náhodných přístupech. HDD se může hodit pro velké archivy nebo zálohy, u nichž je důležitá kapacita a převládá sekvenční přenos.

| Vlastnost             | HDD              | SSD                 |
|-----------------------|------------------|---------------------|
| Princip uložení       | magnetický       | elektronický        |
| Pohyblivé části       | ano              | ne                  |
| Náhodný přístup       | pomalejší        | rychlejší           |
| Hlučnost              | slyšitelný pohyb | tiché               |
| Typická přednost      | kapacita a cena  | rychlost a odolnost |
| Uchování bez napájení | ano              | ano                 |

Table 2: Obecné porovnání vlastností HDD a SSD.

Tabulka 2 zachycuje obecné tendence, nikoliv záruku pro každý konkrétní model. Rychlé HDD může v některé úloze překonat velmi pomalé SSD a výsledný výkon ovlivňuje také použité rozhraní, řadič, velikost požadavků nebo zaplnění zařízení. Při praktickém porovnávání je proto potřeba sledovat parametry a měření konkrétních výrobků.

## 6 Paměťová hierarchie

Žádný běžný druh paměti současně nenabízí nejvyšší rychlost, obrovskou kapacitu, nízkou cenu a trvalé uložení dat. Počítač proto kombinuje několik paměťových úrovní. Blízko procesoru se nacházejí malé a rychlé registry a cache, pod nimi větší hlavní paměť a ještě níže dlouhodobá úložiště. Pro zálohování a archivaci lze přidat síťová úložiště, magnetické pásky nebo jiné systémy s velkou kapacitou, u nichž nevadí delší doba přístupu.

Směrem vzhůru v hierarchii na obrázku 15 roste rychlost a obvykle také cena za uložený bit, zatímco kapacita klesá. Směrem dolů získáváme větší a levnější úložný prostor, ale na data čekáme déle. Jednotlivé úrovně proto plní rozdílné role:

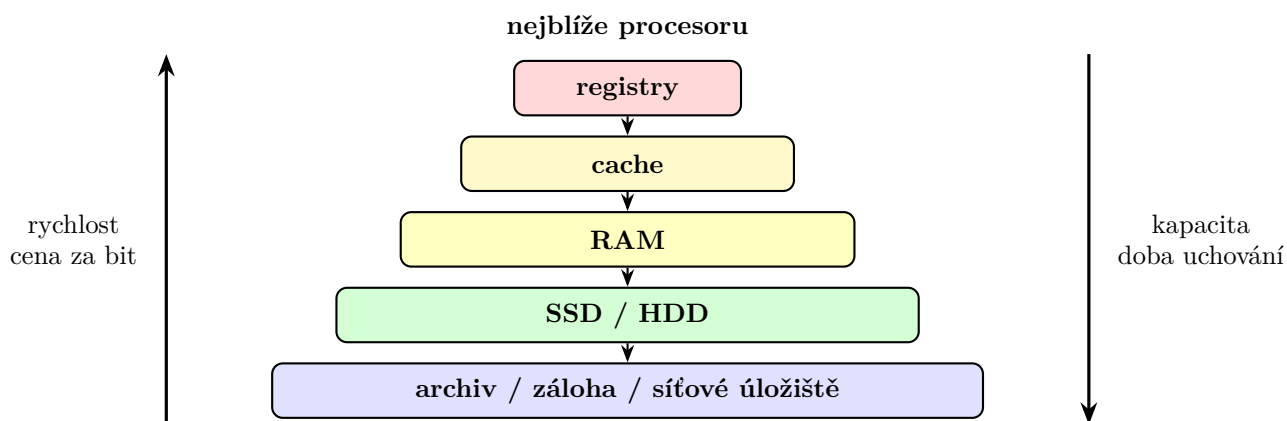


Figure 15: Zjednodušená paměťová hierarchie počítače.

- **registry** uchovávají hodnoty, s nimiž právě pracují instrukce;
- **cache** uchovává kopie nedávno nebo blíže používaných částí paměti;
- **RAM** obsahuje běžící programy a jejich pracovní data;
- **SSD a HDD** dlouhodobě uchovávají programy a soubory;
- **archivní a záložní systémy** upřednostňují kapacitu, cenu a bezpečné uchování před okamžitým přístupem.

| Úroveň        | Relativní rychlost | Relativní kapacita | Volatilní |
|---------------|--------------------|--------------------|-----------|
| Registry      | nejvyšší           | nejmenší           | ano       |
| Cache         | velmi vysoká       | malá               | ano       |
| RAM           | střední            | větší              | ano       |
| SSD/HDD       | nižší              | velká              | ne        |
| Archiv/záloha | nejnižší           | velmi velká        | ne        |

Table 3: Kvalitativní porovnání úrovní paměťové hierarchie.

Paměťová hierarchie funguje dobře tehdy, když se na rychlejších úrovních nacházejí právě ta data, která budou brzy potřeba. Část přesunů zajišťuje hardware automaticky, jako v případě cache. Jiné přesuny řídí operační systém, například když načítá program z SSD do RAM. Další provádí přímo uživatel, když přesune starší data do archivu nebo vytvoří zálohu.

Z pohledu programu může být velká část těchto přesunů skrytá. Program pracuje s adresami v paměti a procesor se pokouší obsloužit požadavky co nejrychleji. Pokud jsou data v registru nebo cache, pokračuje výpočet rychle. Při výpadku cache je potřeba čekat na RAM; pokud se potřebná část programu nebo soubor teprve načítá z úložiště, čekání je ještě delší. Rozdíl mezi jednotlivými úrovněmi vysvětluje, proč má způsob uspořádání dat významný vliv na skutečný výkon.

Paměťová hierarchie vytváří dojem velké a rychlé paměti tím, že kombinuje malou rychlou paměť s většími pomalejšími úrovněmi. Tento dojem je nejpřesvědčivější tehdy, když program vykazuje dobrou časovou a prostorovou lokalitu.

## 6.1 Shrnutí kapitoly

- Procesor vykonává instrukce, ale při své činnosti neustále komunikuje s pamětí, vstupními a výstupními zařízeními.
- Ve von Neumannově architektuře sdílejí instrukce a data společnou paměť; harvardská architektura jejich paměti a přenosové cesty odděluje.
- Základními částmi procesoru jsou výpočetní jednotky, řadič a registry. Instrukce procházejí fázemi načtení, dekódování, získání operandů, provedení a uložení výsledku.
- Cache zkracuje průměrnou dobu přístupu k datům tím, že využívá časovou a prostorovou lokalitu programů.
- Hlavní paměť poskytuje programu lineární adresový prostor, přestože jsou fyzické buňky v čípech uspořádány do matic.
- HDD ukládá data magneticky na rotující plotny, zatímco SSD používá flash paměť bez pohyblivých částí.
- Registry, cache, RAM, úložiště a archivní systémy společně vytvářejí paměťovou hierarchii, která vyvažuje rychlost, kapacitu a cenu.